

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Quiz 2: due on 3/25
  - Homework 9: due on 3/28
  - Lab 3: due on 4/1
  - Project 3: due on 4/11

Previous Lecture …

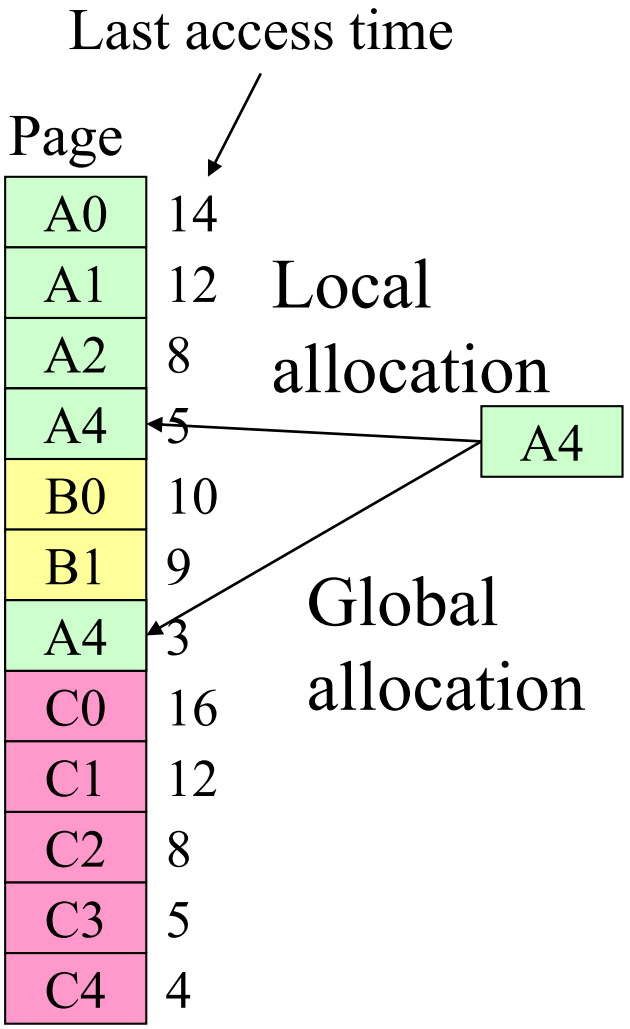
- Tracing page replacement algorithms
  - FIFO and CLOCK
  - LRU and OPT

Muddiest Points

- Do page evictions happen across processes?

Local vs. global allocation policies

- What is the pool of pages eligible to be replaced?
  - Pages belonging to the process needing a new page
  - All pages in the system
- Local allocation: replace a page from this process
  - May be more “fair”: penalize processes that replace many pages
  - Can lead to poor performance: some processes need more pages than others
- Global allocation: replace a page from *any* process

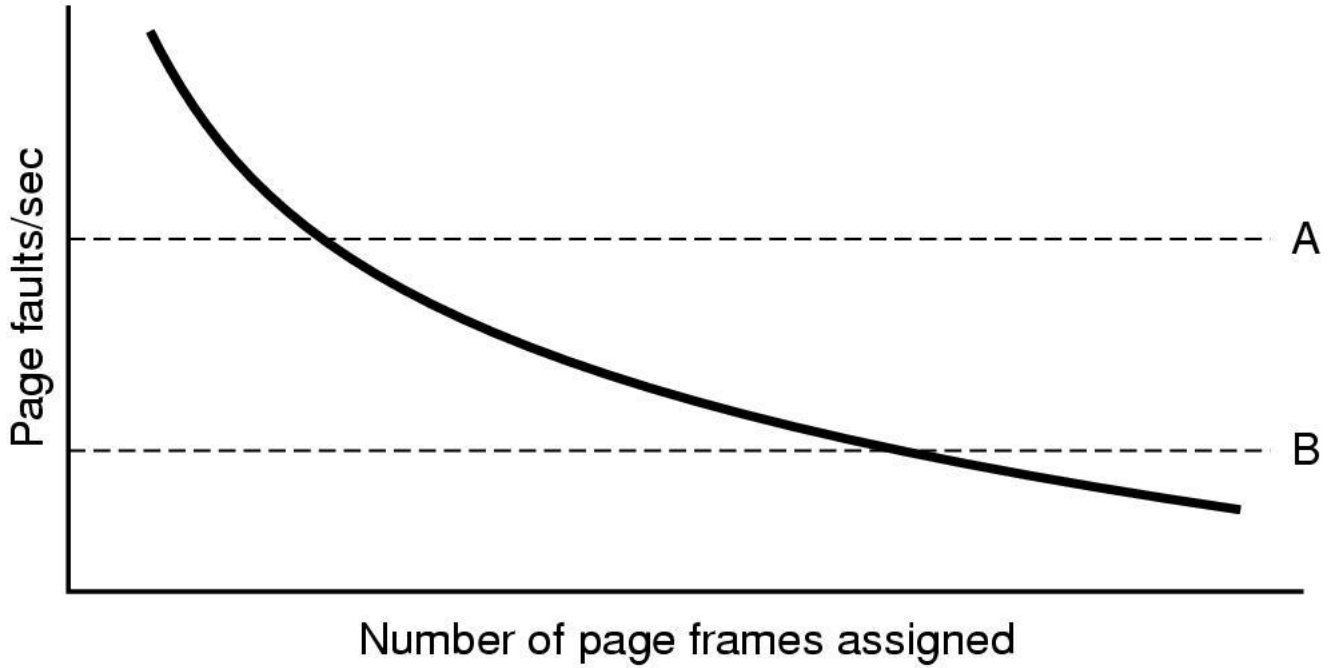


Muddiest Points

- what measures do modern operating systems take in order to minimize thrashing?

Page fault rate vs. allocated frames

- Local allocation may be more “fair”
  - Don’t penalize other processes for high page fault rate
- Global allocation is better for overall system performance
  - Take page frames from processes that don’t need them as much
  - Reduce the overall page fault rate (even though rate for a single process may go up)



Control overall page fault rate

- Despite good designs, system may still thrash
- Most (or all) processes have high page fault rate
  - Some processes need more memory, …
  - but no processes need less memory (and could give some up)
- Problem: no way to reduce page fault rate
- Solution :
  Reduce number of processes competing for memory
  - Swap one or more to disk, divide up pages they held
  - Reconsider degree of multiprogramming

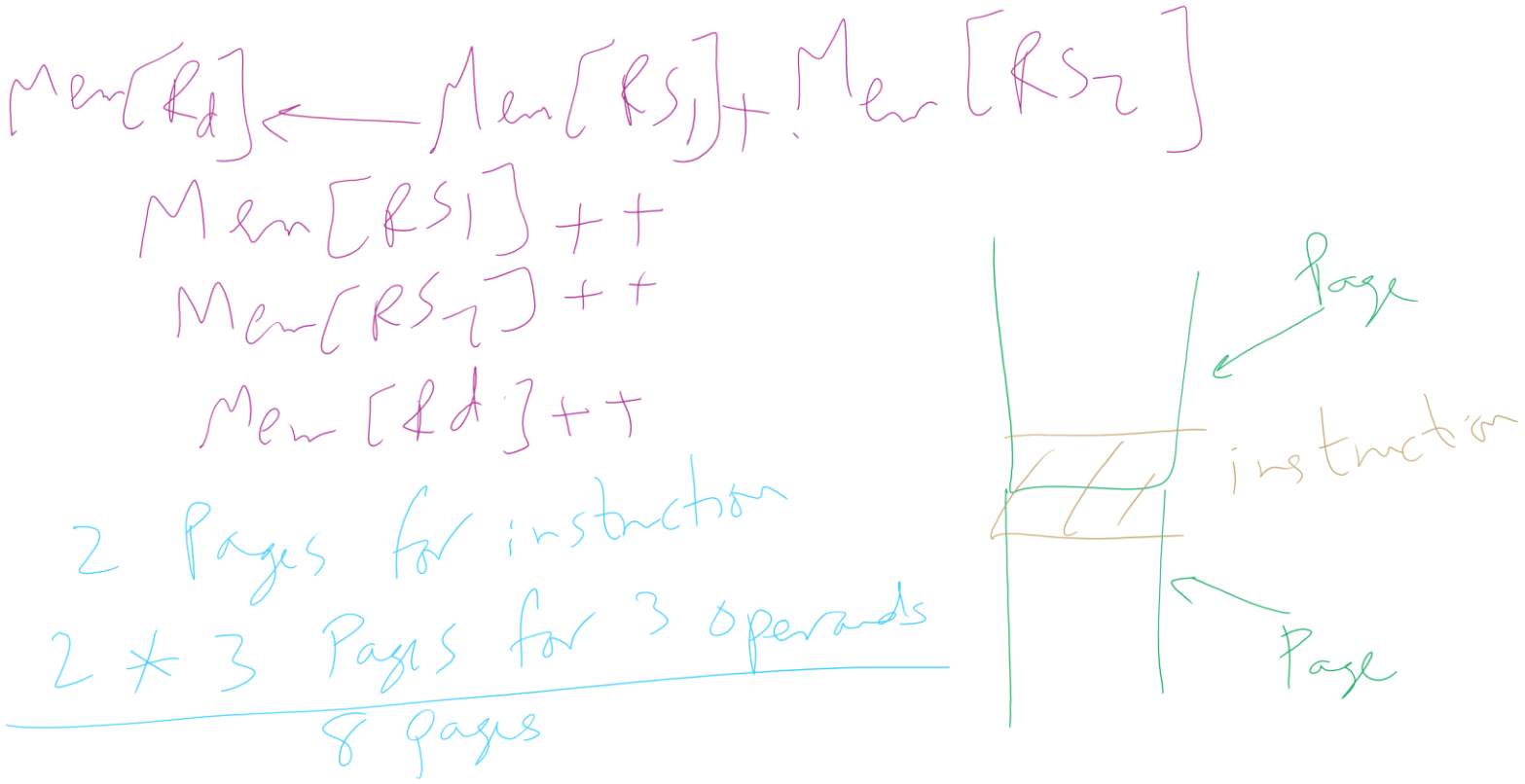
Backing up an instruction

- Problem: page fault happens in the middle of instruction execution
  - Some changes may have already happened
  - Others may be waiting for VM to be fixed
- Solution: undo all of the changes made by the instruction
  - Restart instruction from the beginning
  - This is easier on some architectures than others
- Example: LW R1, 12(R2)
  - Page fault in fetching instruction: nothing to undo
  - Page fault in getting value at 12(R2): restart instruction



Minimum memory allocation to a process

- Example: ADD (Rd)+,(Rs1)+,(Rs2)+
  - Page fault in writing to (Rd): may have to undo an awful lot…



Muddiest Points

- What is disk write?
- What does the * mean?
- CLOCK
  - When does the orbit (or bit?) change in the clock simulation (FIFO/slide 14)
  - how does the clock iterate in the example from class?
- LRU
  - Determining page faults in LRU modelling
  - How to calculate the distance
  - Is there a way to easily determine the disk writes using the LRU stack algorithm?
  - Would you please solve some example for LRU and OPT? Thank you so much

Muddiest Points

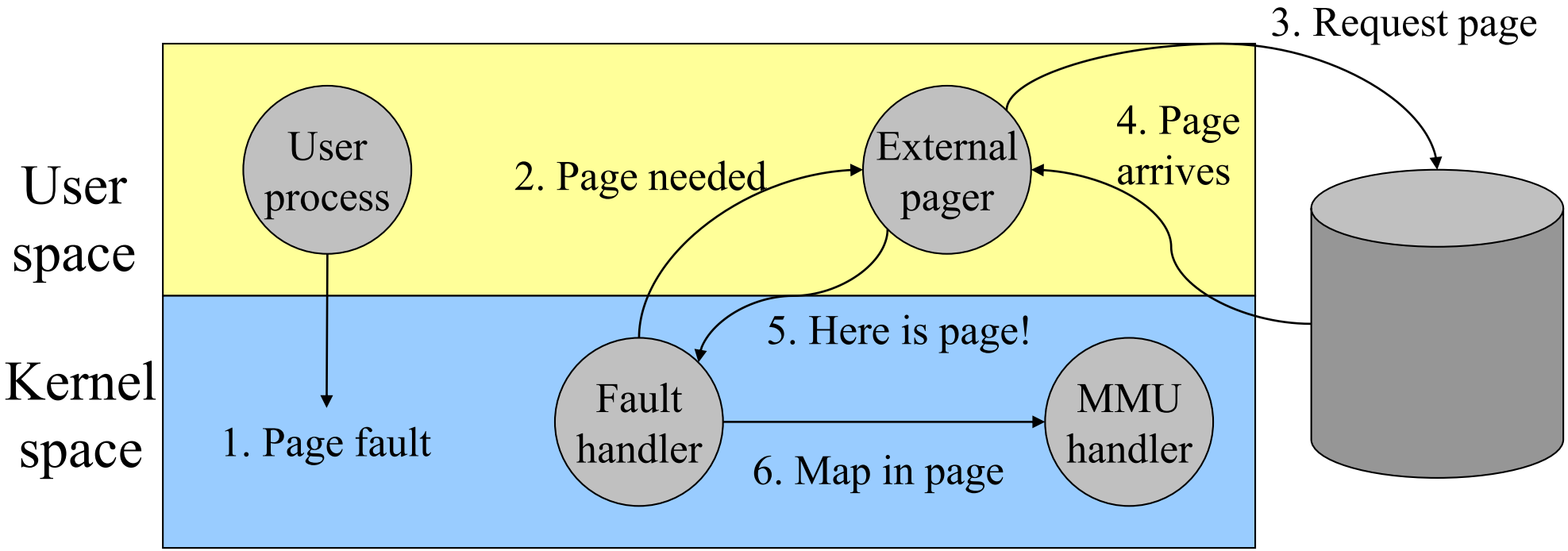
- How the tracing changes between different types of scheduling
- FIFO
  - why does FIFO have more page faults with more memory (small explanation?)?
- How to implement stack algorithm in code

Muddiest Points

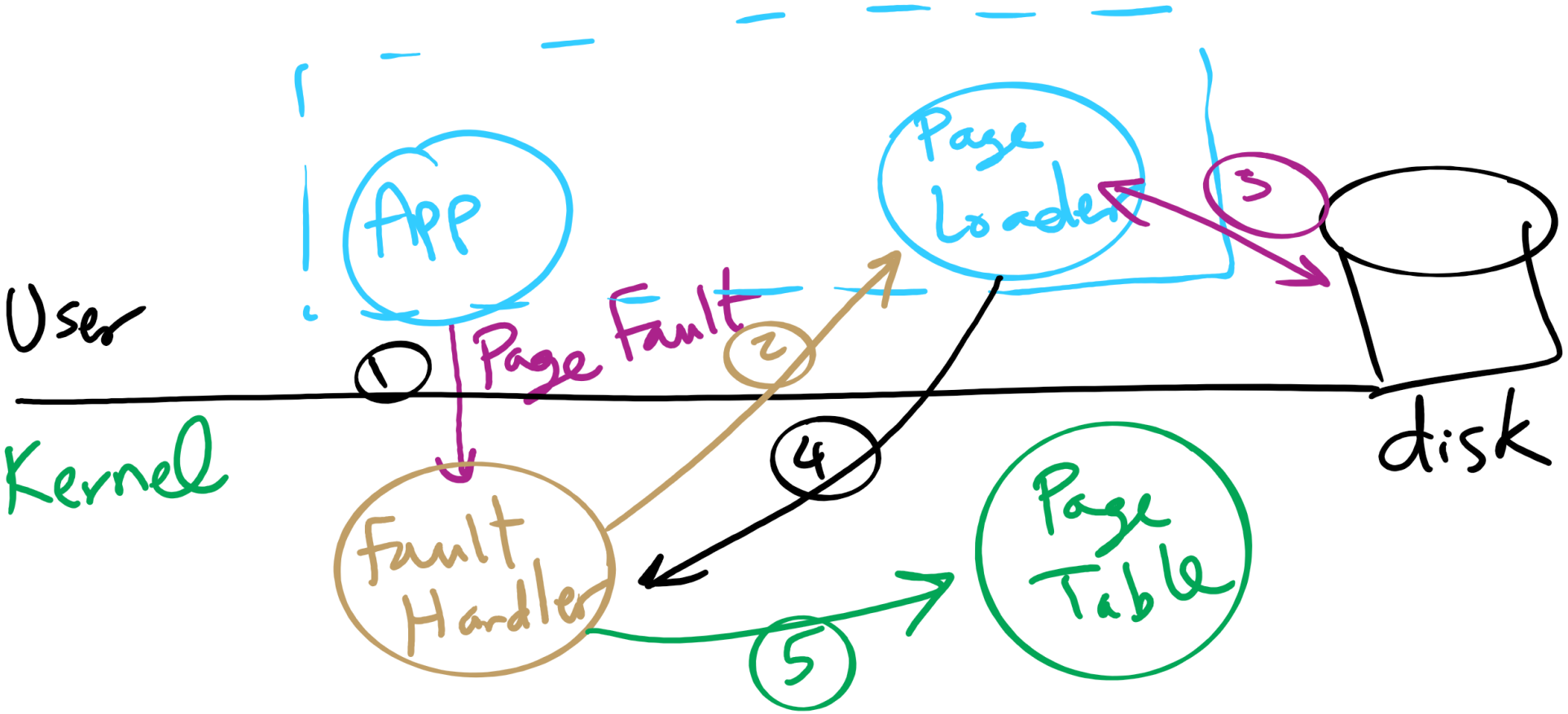
- how to keep track of larger tables that can store more pages.
  - Problem of the Day!

Separating policy and mechanism

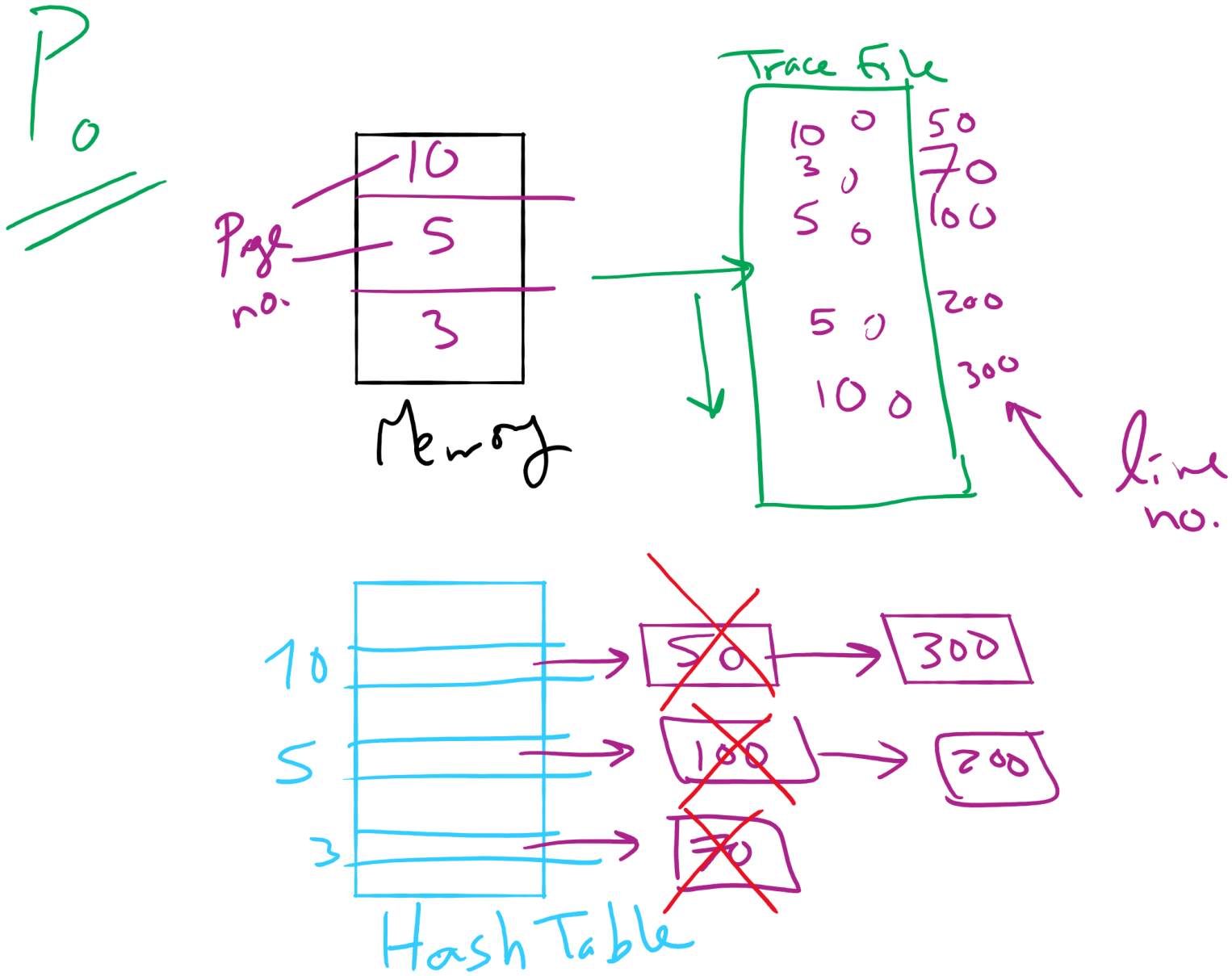
- Mechanism for page replacement has to be in kernel
  - Modifying page tables
  - Reading and writing page table entries
- Policy for deciding which pages to replace could be in user space
  - More flexibility



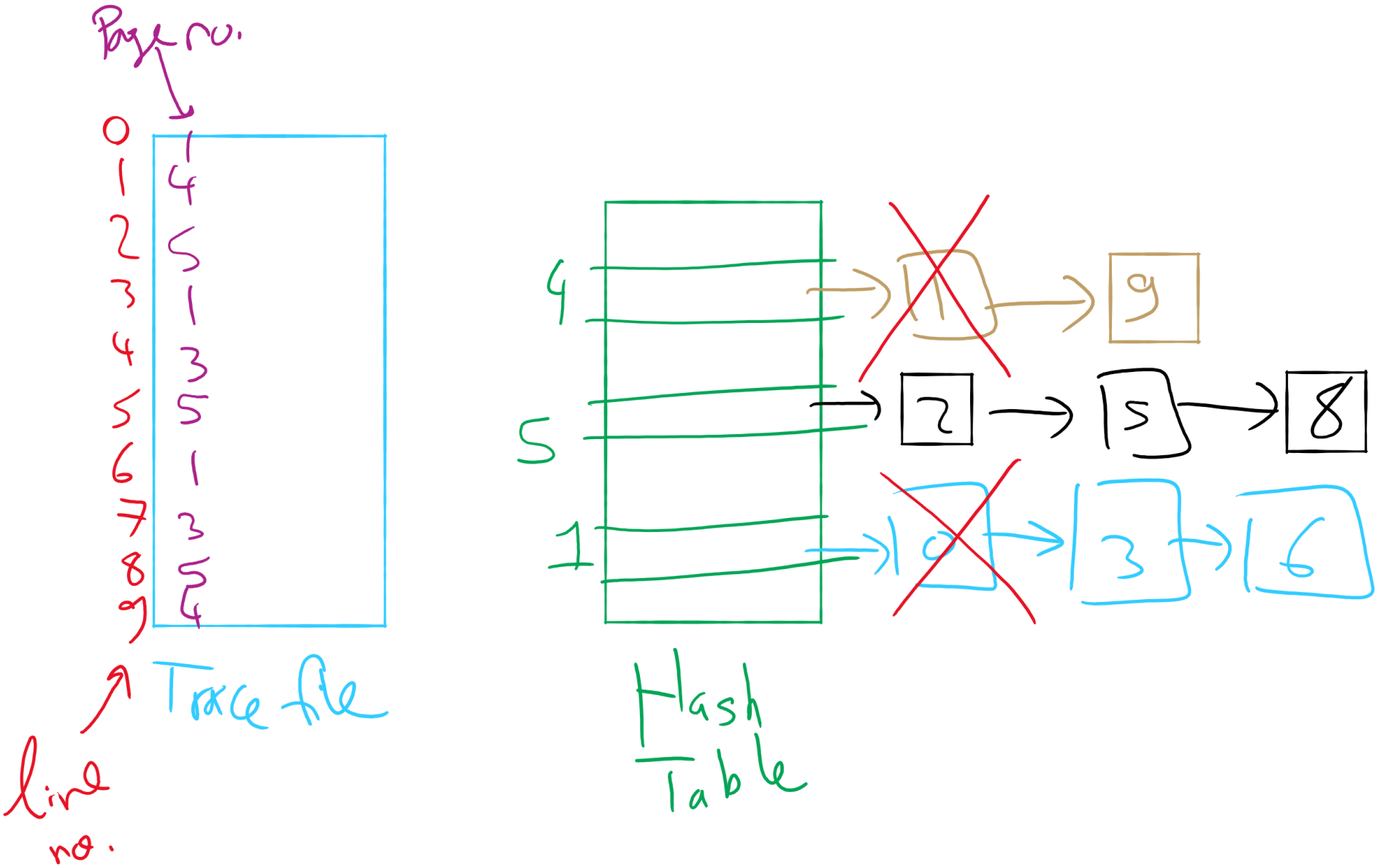
Separating Policy and Mechanism for Page Replacement



Project 3: OPT Simulation

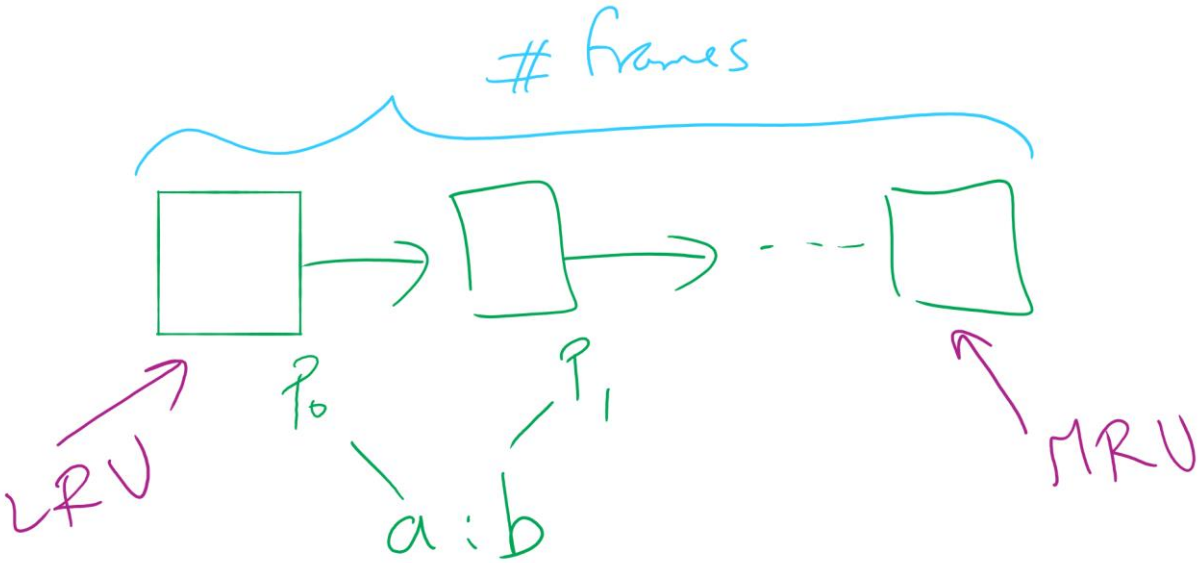


OPT Implementation Example



Project 3: LRU and miscellaneous hints

gunzip 1.trace.gz

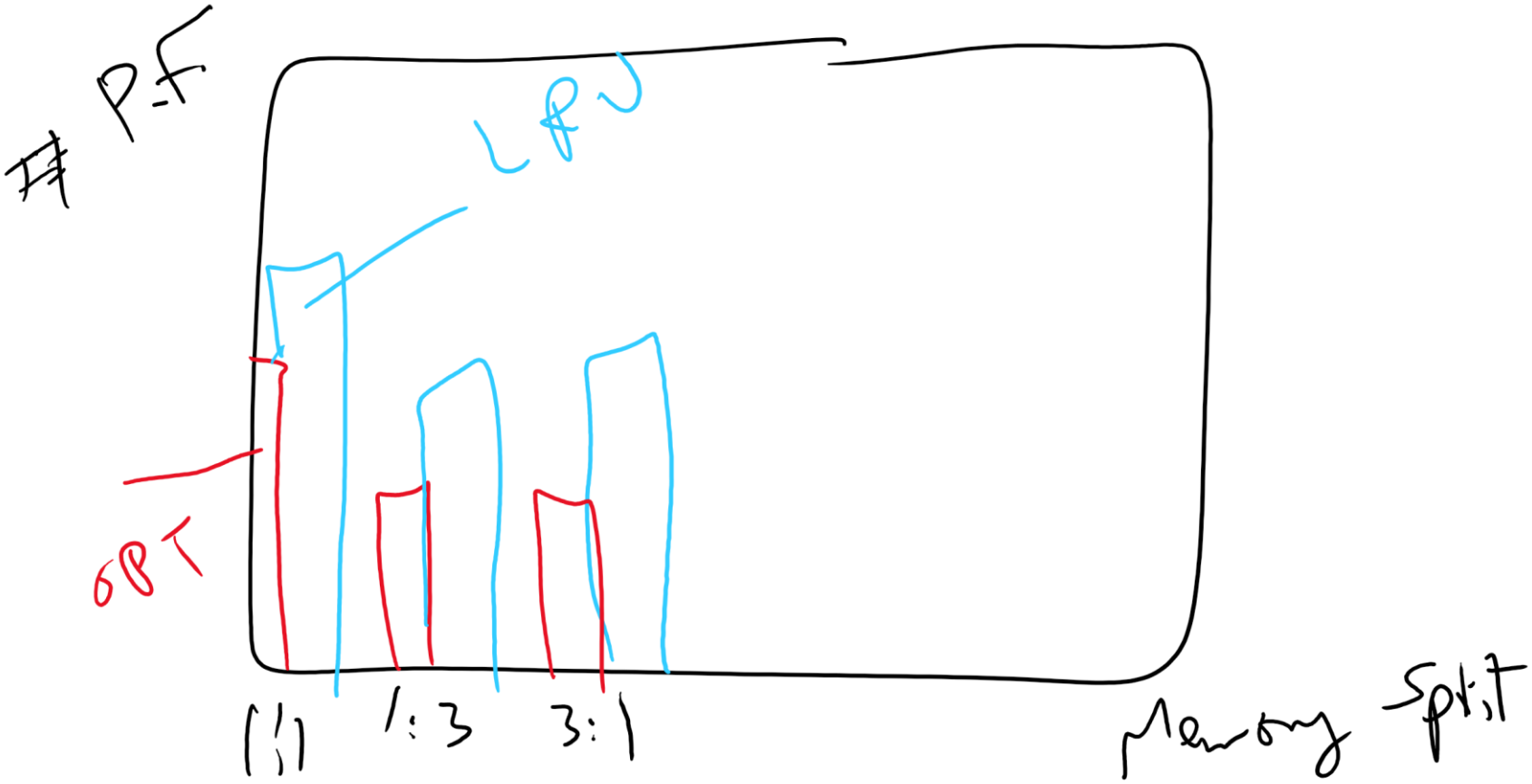


$n \div (a+b) == 0$

$Frames_0 = \left(\frac{n}{a+b}\right) * a$

$Frames_1 = n - Frames_0$

Project 3: Writeup hints



Problem of the Day

- How to keep track of larger page tables that can store more pages
- How big can a page table be?
  - 64-bit machine
  - 4 KB page size
  - How many pages?
  - How many PTE?
  - How big is a PTE?
  - How big is the page table of one process?